

1. 论文名称: Retrieval-Augmented Generation **for** Knowledge-Intensive NLP Tasks
2. 团队: Meta
3. github:  
[https://github.com/huggingface/transformers/tree/main/examples/research\\_projects/rag](https://github.com/huggingface/transformers/tree/main/examples/research_projects/rag)

## Abstract

- 主题:
  - 预训练语言模型在知识密集型NLP任务中的应用限制与提升方法
- 问题:
  - 预训练语言模型在访问和操作知识方面的限制
  - 模型决策来源和知识更新为未解决问题
- 提出方法:
  - 名称: 检索增强生成(RAG)
  - 特点:
    - 结合预训练的参数化和非参数化内存
    - 参数内存为预训练seq2seq模型
    - 非参数内存为Wikipedia的密集向量索引，通过预训练神经检索器访问
- 方法对比:
  - 针对相同与不同文章段落条件的两种RAG配置
- 评估:
  - 在开放领域QA任务上超越现有技术水平
  - 在语言生成任务上，生成更具体、多样化和事实性的语言
- 结果:
  - RAG模型在任务性能和生成语言质量上均优于仅参数化的seq2seq模型

## 1 Introduction

### 1. 研究背景:

- 预训练神经语言模型: 能够学习大量深度知识，无需外部记忆体即可作为参数化隐含知识库。
- 局限性: 难以扩展或修订其记忆，不易提供对预测的洞察，可能产生“幻觉”。

### 2. 混合模型:

- 特点: 结合参数化记忆和非参数化(检索基础)记忆，可以直接修订和扩展知识，检索到的知识可以被检查和解释。
- 例子: REALM和ORQA，结合掩蔽语言模型和可微分检索器，显示出有前景的结果。

### 3. 检索增强生成(RAG):

- **目标:** 将预训练、参数化记忆的生成模型与非参数化记忆结合起来。
- **构建:** RAG模型，参数化记忆为预训练seq2seq模型，非参数化记忆为Wikipedia的密集向量索引，通过预训练的DPR检索器访问。
- **方法:** 结合概率模型端到端训练，DPR提供条件文档，seq2seq模型（如BART）根据这些文档的条件生成输出。

### 4. 先前研究:

- 提出了不同架构，如记忆网络和记忆层，以富化系统。
- 本研究探索了预训练和预加载大量知识的参数化和非参数化记忆组件的设置。

### 5. 研究结果:

- RAG模型在知识密集型任务上表现优异，如Natural Questions、WebQuestions和CuratedTrec，在无约束生成任务上超过了以往的抽取方法。
- 在MS-MARCO和Jeopardy问答生成任务上实验，RAG模型生成更确切、多样化和事实的回答，与最先进的管道模型相比，未经约束的生成性能提高了4.3%，并且非参数化记忆可以用来更新模型的知识，以应对世界的变化。

## 2 Methods

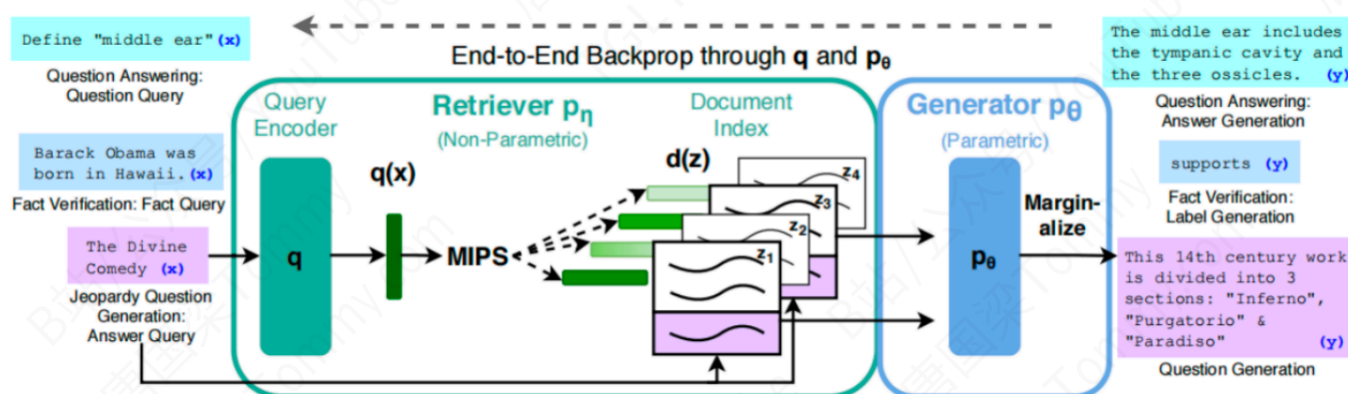


图1: 我们的方法概览。我们结合了一个预训练的检索器（查询编码器 + 文档索引）与一个预训练的序列到序列模型（生成器）并进行端到端的微调。对于查询  $x$ ，我们使用最大内积搜索（MIPS）来找到排名最高的  $K$  个文档  $z_i$ 。对于最终预测  $y$ ，我们将  $z$  视为一个潜变量，并在不同文档中对序列到序列预测进行边缘化。

### 2.1 模型

#### 1. RAG模型探索:

- **输入序列:** 使用输入序列  $x$  检索文本文档  $z$ ，并将其作为生成目标序列  $y$  的额外上下文。

- 组件:

- 检索器  $p_{\eta}(z|x)$ : 带有参数  $\eta$ ，返回给定查询  $x$  的文本段落的分布（Top-K截断）。
- 生成器  $p_{\theta}(y_i|x, z, y_{1:i-1})$ : 生成基于之前  $i - 1$  tokens的当前token  $y_i$ 。

## 2. 训练:

- 将检索到的文档视为潜在变量，端到端训练检索器和生成器。

## 3. RAG模型的两种形式:

### ① RAG-Sequence 模型:

- 使用相同检索文档来生成整个序列。
- 将检索文档作为单个潜在变量，通过Top-K近似得到seq2seq概率  $p(y|z)$ 。
- 公式:  $p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-k}(p(z|x))} p_{\eta}(z|x) p_{\theta}(y|z, x)$

### ② RAG-Token 模型:

- 对每个目标token可以绘制不同的潜在文档，并相应地进行边际化。
- 允许生成器在回答时从多个文档中选择内容。
- 公式:  $p_{\text{RAG-Token}}(y|x) = \prod_i \sum_{z \in \text{top-k}(p(z|x))} p_{\eta}(z|x) p_{\theta}(y_i|z, x, y_1 : i - 1)$

## 4. 应用:

- RAG可用于序列分类任务，此时RAG-Sequence和RAG-Token是等价的。

## 5. 公式解释:

- $p_{\text{RAG-Sequence}}(y|x)$  和  $p_{\text{RAG-Token}}(y|x)$  代表在给定输入  $x$  的情况下生成序列  $y$  的概率。
- $p_{\eta}(z|x)$  是检索器给定输入  $x$  后得到文档  $z$  的概率。
- $p_{\theta}(y_i|z, x, y_{1:i-1})$  是生成器在已知输入  $x$ 、检索到的文档  $z$  和之前生成的tokens  $y_{1:i-1}$  后生成下一个token  $y_i$  的概率。
- Top-K截断是指在所有可能的文档  $z$  中只考虑概率最高的K个。

## >>> 举例讲解 <<<

### 例子场景:

假设我们有一个问题  $x$ ，“首都是什么？”，并且我们的任务是生成答案序列  $y$ ，“柏林是德国的首都。”。

### RAG-Sequence 模型:

### 1. 检索文档:

- 输入序列  $x$  被输入到检索器  $p_{\eta}(z|x)$ 。
- 检索器在数据库（如Wikipedia）中搜索与问题相关的Top-K个文档  $z$ 。比如，它找到了有关德国首都的文章，这个文档记为  $z_1$ 。

### 2. 生成答案:

- 生成器  $p_{\theta}(y|z, x)$  会使用检索到的文档  $z_1$  作为上下文来生成答案。
- 在这个模型中，整个答案序列 "柏林是德国的首都。" 是一次性生成的。

### 3. 公式应用:

- 我们计算  $p_{\eta}(z_1|x)$ ，这是检索器给出文档  $z_1$  的概率。
- 然后，我们计算  $p_{\theta}(y|z_1, x)$ ，这是给定文档  $z_1$  和问题  $x$  时，生成器产生完整答案序列  $y$  的概率。
- 这个过程重复于Top-K个文档，并将每个文档的生成概率相加，得到最终的序列生成概率  $p_{\text{RAG-Sequence}}(y|x)$ 。

## RAG-Token 模型:

### 1. 检索文档:

- 类似于RAG-Sequence模型，但对于答案序列中的每个token  $y_i$ ，都可能使用不同的文档  $z$ 。

### 2. 生成答案:

- 对于答案序列中的每个token "柏林"、"是"、"德国的"、"首都。"，生成器都可能基于不同的文档  $z_i$  来生成。

### 3. 公式应用:

- 对于答案序列中的每个token  $y_i$ ，我们首先计算  $p_{\eta}(z|x)$ ，检索器给出每个可能文档  $z$  的概率。
- 然后计算  $p_{\theta}(y_i|z, x, y_1:i-1)$ ，给定文档  $z$ 、问题  $x$  和之前的tokens  $y_1:i-1$  时，生成器产生当前token  $y_i$  的概率。
- 这个过程对每个token和其可能的文档  $z$  重复进行，并将所有的生成概率乘起来，得到最终的序列生成概率  $p_{\text{RAG-Token}}(y|x)$ 。

在实际操作中，这两个模型的运作会涉及复杂的数学计算和算法优化，但基本思路是：

- RAG-Sequence模型在整个答案序列生成过程中使用同一个文档上下文。
- RAG-Token模型为答案序列中的每个token允许不同的文档上下文，这可能使答案更丰富和准确。

## 2.2 检索器: DPR

检索器  $p_{\eta}(z|x)$  是基于DPR，它采用双编码器结构，其中：

- $d(z)$  是由BERT文档编码器生成的文档的密集表示。
- $q(x)$  是由基于BERT的查询编码器生成的查询表示。

检索过程是计算文档  $z$  和查询  $x$  之间的点积，然后进行指数化和归一化，得到文档的先验概率分布。这个过程可以快速完成，因为它是一个最大内积搜索（MIPS）问题。

例子：

考虑一个查询 "柏林是哪国的首都？"，检索器将：

1. 将查询编码成  $q(x)$ 。
2. 在预建立的文档索引中查找最相关的文档  $z$ ，这些文档的表示为  $d(z)$ 。
3. 计算每个文档与查询的点积  $\exp(d(z)^T q(x))$ 。
4. 选择点积值最高的Top-K个文档。

## 2.3 生成器: BART

生成器  $p_\theta(y_i|x, z, y_1:i-1)$  基于BART-large，一个预训练的seq2seq变换器模型，用于生成目标序列。当结合输入  $x$  和检索内容  $z$  时，BART通过简单的串联它们来生成文本。

例子：

如果检索到的文档  $z$  包含信息 "柏林是德国的首都"，生成器将：

1. 将此信息与查询  $x$  结合。
2. 生成答案序列，如 "柏林是德国的首都。"。

## 2.4 训练

检索器和生成器是共同训练的，不需要直接监督来指定应检索哪个文档。训练过程包括：

- 使用输入/输出对  $(x_j, y_j)$  的微调训练语料库。
- 最小化每个目标的负边际对数似然。
- 使用Adam优化器进行随机梯度下降。

## 2.5 解码

在测试时，RAG-Sequence和RAG-Token采用不同的近似方法来解码最可能的序列  $y$ 。

**RAG-Token：**

- 可以看作是标准的自回归seq2seq生成器，用传统的beam解码器插入转换概率。

**RAG-Sequence：**

- 使用一种"彻底解码"的方法，它在整个文档集上运行额外的前向传递来估计序列的概率，这对于长输出序列是计算密集的。
- 也有一种"快速解码"的方法，避免了在候选集  $Y$  生成后运行额外的前向传递。

例子：

假设我们想生成回答 "柏林是德国的首都。":

- 对于 **RAG-Token**，我们为每个token（例如 "柏林"、"是"、"德国的"、"首都。"）分别检索文档，并用beam搜索来选择最佳的token。
- 对于 **RAG-Sequence**，我们会对整个答案序列运行一次beam搜索，并对所有可能的文档组合评分，选择最有可能的完整答案序列。

## 3 实验

### 3.1 开放域问答

- **实验设置**: 使用单一的Wikipedia dump作为非参数化知识源，分为21M文档。
- **检索器训练**: 使用文档编码器建立MIPS索引，训练时检索Top- $\{5, 10\}$ 文档，测试时根据开发数据集确定  $k$  值。
- **任务**: 比较RAG与传统的抽取式QA模式，以及不利用检索而直接生成答案的方法。
- **数据集**: Natural Questions (NQ), TriviaQA (TQA), WebQuestions (WQ), 和 CuratedTrec (CT)。

### 3.2 抽象式问答

- **RAG模型**: 能够执行超越简单抽取式QA的抽象式文本生成。
- **任务**: 在MSMARCO数据集上进行开放域抽象式问答任务。

### 3.3 危险问题生成

- **任务**: 生成Jeopardy类型的问题，这是一个比标准开放域QA任务更具挑战性的知识密集型生成任务。
- **评估方法**: 使用Q-BLEU和Q-BLEU1指标，以及两种人类评估标准来评估问题生成的事实性和特异性。

### 3.4 事实验证

- **任务**: 使用FEVER数据集，分类自然语言声明是否由Wikipedia支持或反驳。
- **方法**: 将FEVER分类问题映射到单个输出token，并直接训练RAG模型。

## 结论

- RAG模型能够在多种知识密集型任务中表现出色，包括开放域问答、抽象式问答、问题生成和事实验证。
- 实验结果表明，RAG模型在这些任务上能够依靠检索到的知识以及参数化的知识生成准确和合理的响应。

## 4 结果

### 4.1 开放域问答

- **结果:** RAG在所有四个开放域QA任务中均设立了新的艺术水平。
- **与其他模型比较:** RAG结合了“闭书”（仅参数化）方法的生成灵活性和“开书”检索式方法的性能。
- **优势:** RAG可以生成答案，即使在可能从文档中抽取答案的情况下也是如此。
- **实验细节:** RAG的检索器使用DPR进行初始化，利用Natural Questions和TriviaQA上的检索监督。

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

|        | Model           | NQ          | TQA               | WQ          | CT          |
|--------|-----------------|-------------|-------------------|-------------|-------------|
| Closed | T5-11B [52]     | 34.5        | - / 50.1          | 37.4        | -           |
| Book   | T5-11B+SSM [52] | 36.6        | - / 60.5          | 44.7        | -           |
| Open   | REALM [20]      | 40.4        | - / -             | 40.7        | 46.8        |
| Book   | DPR [26]        | 41.5        | <b>57.9</b> / -   | 41.1        | 50.6        |
|        | RAG-Token       | 44.1        | 55.2/66.1         | <b>45.5</b> | 50.0        |
|        | RAG-Seq.        | <b>44.5</b> | 56.8/ <b>68.0</b> | 45.2        | <b>52.2</b> |

### 4.2 抽象式问答

- **结果:** RAG-Sequence在Open MS-MARCO NLG上比BART表现出更好的性能，增加了2.6个Bleu分数和2.6个Rouge-L分数。

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] \*Uses gold context/evidence. Best model without gold access underlined.

| Model    | Jeopardy    |             | MSMARCO      |              | FVR3        | FVR2         |
|----------|-------------|-------------|--------------|--------------|-------------|--------------|
|          | B-1         | QB-1        | R-L          | B-1          | Label       | Acc.         |
| SotA     | -           | -           | <b>49.8*</b> | <b>49.9*</b> | <b>76.8</b> | <b>92.2*</b> |
| BART     | 15.1        | 19.7        | 38.2         | 41.6         | 64.0        | 81.1         |
| RAG-Tok. | <b>17.3</b> | <b>22.2</b> | 40.1         | 41.5         | 72.5        | <u>89.5</u>  |
| RAG-Seq. | 14.7        | 21.4        | <u>40.8</u>  | <u>44.2</u>  |             |              |



## 4.3 危险问题生成

- 结果: RAG-Token在Jeopardy问题生成上表现优于RAG-Sequence，两者都比BART有更好的性能。
- 评估: 评价者表明，RAG在42.7%的案例中更具事实性，而BART则在7.1%的案例中更准确。

Table 3: Examples from generation tasks. RAG models generate more specific and factually accurate responses. ‘?’ indicates factually incorrect responses, \* indicates partially correct responses.

| Task                         | Input                            | Model | Generation                                                                                     |
|------------------------------|----------------------------------|-------|------------------------------------------------------------------------------------------------|
| MS-MARCO                     | define middle ear                | BART  | ?The middle ear is the part of the ear between the middle ear and the nose.                    |
|                              |                                  | RAG-T | The middle ear is the portion of the ear internal to the eardrum.                              |
|                              |                                  | RAG-S | The middle ear includes the tympanic cavity and the three ossicles.                            |
|                              | what currency needed in scotland | BART  | The currency needed in Scotland is Pound sterling.                                             |
|                              |                                  | RAG-T | Pound is the currency needed in Scotland.                                                      |
|                              |                                  | RAG-S | The currency needed in Scotland is the pound sterling.                                         |
| Jeopardy Question Generation | Washington                       | BART  | ?This state has the largest number of counties in the U.S.                                     |
|                              |                                  | RAG-T | It's the only U.S. state named for a U.S. president                                            |
|                              |                                  | RAG-S | It's the state where you'll find Mount Rainier National Park                                   |
|                              | The Divine Comedy                | BART  | *This epic poem by Dante is divided into 3 parts: the Inferno, the Purgatorio & the Purgatorio |
|                              |                                  | RAG-T | Dante's "Inferno" is the first part of this epic poem                                          |
|                              |                                  | RAG-S | This 14th century work is divided into 3 sections: "Inferno", "Purgatorio" & "Paradiso"        |

Document 1: his works are considered classics of American literature ... His wartime experiences formed the basis for his novel "A Farewell to Arms" (1929) ...

Document 2: ... artists of the 1920s "Lost Generation" expatriate community. His debut novel, "The Sun Also Rises", was published in 1926.

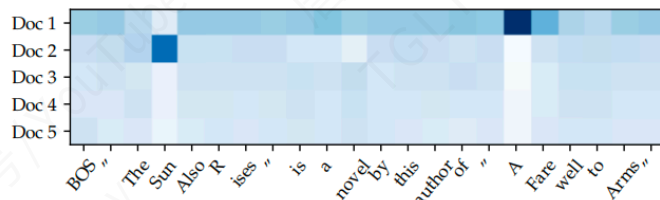


Figure 2: RAG-Token document posterior  $p(z_i | x, y_i, y_{-i})$  for each generated token for input "Hemingway" for Jeopardy generation with 5 retrieved documents. The posterior for document 1 is high when generating "A Farewell to Arms" and for document 2 when generating "The Sun Also Rises".

## 4.4 事实验证

- 结果: RAG在FEVER的3-way分类中，与使用域特定架构和大量工程训练的最新技术模型相比，分数落后4.3%。

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] \*Uses gold context/evidence. Best model without gold access underlined.

| Model    | Jeopardy    |             | MSMARCO      |              | FVR3        | FVR2         |
|----------|-------------|-------------|--------------|--------------|-------------|--------------|
|          | B-1         | QB-1        | R-L          | B-1          | Label       | Acc.         |
| SotA     | -           | -           | <b>49.8*</b> | <b>49.9*</b> | <b>76.8</b> | <b>92.2*</b> |
| BART     | 15.1        | 19.7        | 38.2         | 41.6         | 64.0        | 81.1         |
| RAG-Tok. | <b>17.3</b> | <b>22.2</b> | 40.1         | 41.5         | 72.5        | <u>89.5</u>  |
| RAG-Seq. | 14.7        | 21.4        | <u>40.8</u>  | <u>44.2</u>  |             |              |



## 4.5 其他结果

- **生成多样性:** RAG在Jeopardy问题生成上显示出比BART更高的事实性和特定性。
- **检索消融:** 显示RAG学会检索任务相关信息的关键特征。
- **索引热交换:** RAG的知识可以在测试时通过替换其非参数化内存轻松更新。
- **检索更多文档的效果:** 训练模型时检索5个或10个文档，测试时调整检索文档的数量会影响性能。

Table 5: Ratio of distinct to total tri-grams for generation tasks.

|           | MSMARCO | Jeopardy QGen |
|-----------|---------|---------------|
| Gold      | 89.6%   | 90.0%         |
| BART      | 70.7%   | 32.4%         |
| RAG-Token | 77.8%   | 46.8%         |
| RAG-Seq.  | 83.5%   | 53.8%         |

Table 6: Ablations on the dev set. As FEVER is a classification task, both RAG models are equivalent.

| Model               | NQ          | TQA         | WQ          | CT          | Jeopardy-QGen |             | MSMarco     |             | FVR-3          | FVR-2       |
|---------------------|-------------|-------------|-------------|-------------|---------------|-------------|-------------|-------------|----------------|-------------|
|                     |             | Exact Match |             |             | B-1           | QB-1        | R-L         | B-1         | Label Accuracy |             |
| RAG-Token-BM25      | 29.7        | 41.5        | 32.1        | 33.1        | 17.5          | 22.3        | 55.5        | 48.4        | <b>75.1</b>    | <b>91.6</b> |
| RAG-Sequence-BM25   | 31.8        | 44.1        | 36.6        | 33.8        | 11.1          | 19.5        | 56.5        | 46.9        |                |             |
| RAG-Token-Frozen    | 37.8        | 50.1        | 37.1        | 51.1        | 16.7          | 21.7        | 55.9        | 49.4        | 72.9           | 89.4        |
| RAG-Sequence-Frozen | 41.2        | 52.1        | 41.8        | 52.6        | 11.8          | 19.6        | 56.7        | 47.3        |                |             |
| RAG-Token           | 43.5        | 54.8        | <b>46.5</b> | 51.9        | <b>17.9</b>   | <b>22.6</b> | 56.2        | <b>49.4</b> | 74.5           | 90.6        |
| RAG-Sequence        | <b>44.0</b> | <b>55.8</b> | 44.9        | <b>53.4</b> | 15.3          | 21.5        | <b>57.2</b> | 47.5        |                |             |

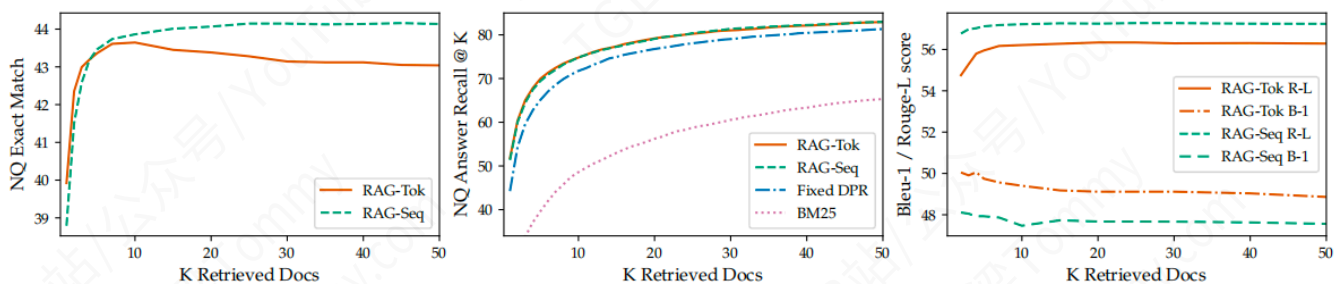


Figure 3: Left: NQ performance as more documents are retrieved. Center: Retrieval recall performance in NQ. Right: MS-MARCO Bleu-1 and Rouge-L as more documents are retrieved.

## 5 相关工作

### 5.1 单任务检索

- 先前研究显示检索能提高多种NLP任务的性能，包括开放域问答、事实核查、事实完成、长篇问答、维基百科文章生成、对话、翻译和语言建模。

- 提出的工作将这些在单个任务中整合检索的成功案例统一起来，展示了单一检索基础架构能跨多个任务实现强大的性能。

## 5.2 通用架构

- 以往的工作集中于不使用检索的通用NLP架构，但展示了单一预训练语言模型在多任务上的成功。
- RAG模型旨在扩展可以使用单一统一架构解决的任务范围，通过双向注意力来增强判别和生成任务的性能。

## 5.3 学习检索

- 有相当多的工作致力于学习信息检索，尤其是与预训练的神经语言模型一起使用。
- 本工作展示了一个单一的检索基础架构可以针对多种任务进行微调，实现强大的性能。

## 5.4 基于记忆的架构

- 文档索引可以被视为大型的外部记忆，类似于记忆网络。
- 这种记忆的关键特点是它由原始文本组成，使得记忆对人类可读且可以编辑，为知识密集型任务提供了事实嵌入。

## 5.5 检索和编辑方法

- RAG方法与检索和编辑风格的方法有一定的相似性，其中检索出的输入-输出对用于生成最终的输出。
- 这类方法在多个领域取得了成功，RAG模型在这些设置中可能代表未来的研究方向。

## 6 讨论

- 展示了具有参数化和非参数化记忆访问的混合生成模型。
- RAG模型在开放域问答任务上取得了最先进的结果。
- 人们倾向于选择RAG生成的结果，因为它比纯参数化的BART更为准确和具体。
- 检索组件的有效性得到验证，且检索索引可以在不需要任何重新训练的情况下进行热交换更新模型。
- 未来工作可能探索两个组件是否可以从头开始联合预训练，以及如何最有效地结合参数化和非参数化记忆，应用于广泛的NLP任务。